302 Summer 2013
Project 3

Due Date: ***Friday July 19 midnight.***

Possible Points: 100 (+10)

V 4.0 (7.12.2013 – 11.30)

Note:
This project is ***significantly*** more involved than the first two, and it will require a great deal more effort from your part. Please start as soon as possible.

Working in groups possibility:
If by Tuesday July 10 you think that you would do better in a group for this project, and you have peers that feel the same, you can form groups of up to three for this project. However there will be a catch:

a) Group submissions will not be getting any extra credit points.
b) Group submissions will be capped at 85% for groups of two, and at 70% for groups of three.
   i.e. If you are in a group of and your submission is worth 68% you will all get the full points earned: 68/100 in the case of groups of two and 68/100 in the case of groups of three.
   If your submission is perfect though and worth 100% you will all get up to the cap, i.e. you will all get 85/100 in the case of groups of two and 70/100 in the case of groups of three.
   Of course this means that you should be careful in deciding to enter a group. You should be willing to enter one ONLY if you think that there is no way that you could do better than 85 or 70 respectively.

   Note that the group will have to be approved by me (i.e. you should contact me and tell me who is in the group with who and who will be doing what.) In other words if you form a group and you tell me that person A will do a couple of classes and person B everything else, then the group will not be approved.
   The authorship of every class and method in a group submission should be clearly stated in the class or method header.
   After the submission, the groups will have to meet with me and explain who did what and how they did it.
   If someone in a group is found to have done less work than the rest, then that person will receive a lower grade than the rest of the group. If someone in a group is found to have done no work on their own whatsoever, **the entire group** will receive a zero.
c) Group submissions will not be open for re-submission after the grade release.

In other words, I would like to give you the option to form groups because some people feel more comfortable working and studying with others, or they might feel intimidated by the scope of the project, but I do not want to give you an advantage for working in a group. You have the choice, but you do not need to use it. If you feel that you do need to work in a group but you cannot find someone to form one, just forget about it. It is not a big deal. This is NOT a group project. It is meant to be doable by a single person that knows precisely the things that you know about Java.

Background

In this problem we will create a simulation of the Monty Hall Problem which is based on an old US show called "L*et's make a deal.*"

Here is the basic setup:
There is a host (named Monty Hall), a player and 3 closed doors. One of the doors has a large prize behind it. The other two have nothing. The player gets to pick a door. After the player picks a door, Monty [who knows which door has the prize behind it] opens one of the non prize doors among the remaining doors. Then Monty asks the player if she would like to switch her pick.

The question then is :

- Should the player switch at this point ?
- Should the player stick with her initial choice?
- Does it even matter?

It turns out that the player should ALWAYS switch their choice no matter what. I will explain in class why this is so, and alternatively you can investigate the why on your own.
However it is the case that most people no matter what, they simply do not believe that there is a strategy that wins more often than not: namely the strategy of always switching if Monty offers you an alternative.

To convince such people that indeed if you switch you will win more often than not, we will create a simulation of the game and we will run it a number of times, with a number of different players that they will have different strategies. At the end we can simply check who has won the most money overall. In the process of creating the simulation we will use everything we have discussed in class so far: Inheritance, overloading, arrays, loops, branching etc.
In order to be able to complete this project you will also have to use the debugging capabilities of eclipse much more extensively than before.

The Simulator

Our simulator will consist of all 3 “types” of classes that we talked so far: classes that are describing concepts, classes that are simply a collection of methods, and a class that is runnable. Furthermore it will be an extension of the problem. So instead of simulating just one player and one play, with only 3 doors, we will be able to have an arbitrary number of doors, and arbitrary number of players, and repeat the game an arbitrary number of times.

Concept Classes:

Percentage

You will create a class *Percentage* inside package *cs302.montyhall.concepts.abstractconcepts*

A percentage is a floating point number between 0 and 100 inclusive. So to represent a percentage you need one instance variable of type float.

The class will have the following public methods:

- A constructor that takes as a parameter a *float* value and if the value is between 0 and 100 inclusive it will set the instance variable to that value. Otherwise if the value passed was larger than 100 it will set the instance variable to 100, and if the value passed was smaller than 0 it will set the instance variable to 0.

- A copy constructor that takes as a parameter another *Percentage* object. It then uses the values stored in that object to initialize its own values.

- A *getPercentage* method that takes no arguments and returns the value of the instance variable.

- A *setPercentage* method that takes an argument of value *float* and sets the instance variable appropriately as described in the constructor,i.e. a percentage cannot be less than 0 or more than 100. It returns nothing.

- A *toString* method that takes no arguments and return a *String* representation of the percentage as follows: <percentage value>%
  e.g. if a percentage value is 76.4 the to String will return the string :
  76.4%

The class *Percentage* will have **NO other public methods or variables**.

Strategy

You will create a class *Strategy* inside package *cs302.montyhall.concepts.abstractconcepts*

A strategy is a percentage that simulates how often we would like to switch our choice, plus a name (e.g. "I always switch"). You should create appropriate instance variables to hold this information. Your

Strategy class will have the following public methods:

- A constructor that takes as a parameter a *Percentage* that denotes the frequency of switching and a *String* that gives a name to our strategy, and is using these values to initialize its instance variables.

- A *getHowOftenTheStrategySwitches* that takes no arguments and returns an object of type *Percentage* that represents how often this strategy is switching.

- A *setHowOftenTheStrategySwitches* that takes a *Percentage* object as an argument and sets the appropriate instance variable. It returns nothing.

- A *getStrategyName* that returns a *String* representing the name of the strategy.

- A *setStrategyName* that gets a *String* as an argument and sets the appropriate instance variable. It returns nothing.

- A *toString* method that takes no arguments and returns a *String* that describes the strategy: "<whatever the strategy name is>": a strategy in which we switch <the float the represents the how often>% of the time."
  For example a strategy that switches 51.6% of the time and is named "George's strategy" would print:
  "George's strategy": a strategy in which we switch 51.6% of the time.

The class *Strategy* will have **NO other public methods or variables**.

Door

You will create a class *Door* inside package *cs302.montyhall.concepts.concreteconcepts*

The class Door will define a door object using a single integer which will be the door's id and will have the following public methods:

- A constructor that takes a single int argument and initializes the Door object appropriately.
- A getID method that takes no arguments and returns the appropriate instance variable.
- A toString method that returns the id of the door as a string.
- An equals method that takes as an argument another Door object and returns true if the two doors are equal and false otherwise. Two doors are equal if they have the same id.

The class Door should have **NO other public methods or variables**.

MontyDoor

You will create a class MontyDoor inside package cs302.montyhall.concepts.concreteconcepts

The class MontyDoor will extend the class Door. It will have an additional instance variable (boolean type) that will keep track of whether there is a prize behind the MontyDoor.

The class MontyDoor will have the following public methods:

- A constructor that will take two parameters, a boolean and an int and will initialize the appropriate instance variables.

- A hasPrize method that takes no arguments and returns a boolean that is true if the door has a prize behind it and false otherwise.

- An equals method that takes takes as an argument another MontyDoor object and returns true if the two doors are equal and false otherwise. Two MontyDoors are equal if they have the same id and both have a prize.

- A toString method that takes no arguments and returns a String representation of the door as follows:
  If the door has a prize behind it:
  Door <doors id> has the PRIZE!!
  If the door does not have a prize behind it:
  Door <door id> has no prize…
  e.g. if door with id 16 has a prize behind it the toString would return the string:
  Door 16 has the PRIZE!!

Since MontyDoor extends Door, you should NOT duplicate variables, or methods that already exists in Door. Doing so will result in **0 points for this part of the implementation**. Furthermore this class should contain **NO more public methods or variables**.

AudienceMemberPlayer

You will create a class *AudienceMemberPlayer* inside package *cs302.montyhall.concepts.concreteconcepts*

An *AudienceMemberPlayer* is a potential player. It is a person in the audience that could be picked up by Monty, to play the game. A potential player among the audience should have a name, and a Strategy. Create the *AudienceMemberPlayer* with the appropriate instance variables and the following public methods:

- A constructor that takes two arguments, one of type *String* and one of type *Strategy*, and initializes the appropriate instance variables.

- Two methods called *getPlayersName* and *getPlayersStrategy* that take no parameters and return the value of the appropriate instance variable.

- A *setPlayersStrategy* method that takes one argument of type *Strategy* and sets the appropriate variable.

- A *toString* method that takes no arguments and returns a *String* representation of the *AudienceMemberPlayer* as follows:
  <the name of the player>'s strategy is <the toString representation of the players strategy here>
  e.g. suppose that AudienceMemberPlayer named George has a Strategy of switching 40% of the time that he named "40tude".
  Then we would get:
  George's strategy is "40tude": a strategy in which we switch 40.0% of the time.

The *AudienceMemberPlayer* class will have **NO other public methods or variables**.

MontyRandomness

Here is a class where you will have to simulate something so abstract as the concept of randomness.
You will create a class *MontyRandomness* inside package *cs302.montyhall.concepts.abstractconcepts*

A *MontyRandomness* is a class that has a single instance variable of type *Random*, which in our case will be our randomness provider, in the same sense that *Scanner* was our user's input reader so far.
You will have to import the *java.util.Random* in order to be able to use objects of type *Random*.

The class *MontyRandomness* will have the following public methods:

- A constructor that will take an argument of type *long*, and then:
  - 1st initializes its instance variable using the default constructor of *Random*.
  - Then calls a *reset* method (described below) that resets the seed of the instance variable using the passed *long* parameter.

- A *reset* method that takes an argument of type *long* and sets the seed of the instance variable using the value of the passed *long* parameter. This method will not return anything.

- A *getRandom* method that takes no arguments and returns the value of the instance variable.

- A *getARandomDoubleBetweenTheTwoIndexes* method that takes as arguments two parameters of type *int*, and returns a random value of type *double* within the range specified by the two *int* parameters.
  e.g.
  `getARandomDoubleBetweenTheTwoIndexes(0, 5)`
  could return anything from 0.0 to 5.0 inclusive, i.e. one among (0.0, …2.34,…4.85,..,5.0)

- A *getARandomIntBetweenTheTwoIndexes* method that takes as arguments two parameters of type *int*, and returns a random value of type *int* within the range specified by the two *int* parameters.
  e.g.
  `getARandomIntBetweenTheTwoIndexes (2, 5)`
  could return anything from 2 to 5 inclusive, i.e. one among (2,3,4,5)

- A *getARandomIntBetween0And* method that takes one argument of type *int* and returns a random value of type *int* within the range specified by 0 and that argument.
  e.g.
  `getARandomIntBetween0And(5)`
  could return anything from 0 to 5 inclusive, i.e. one among (0,1,2,3,4,5)

You may assume that the parameters passed in your methods are always valid (no non-numbers, no negative numbers, etc.).
Your *MontyRandomness* should **have NO more public methods or variables**, and it should reuse code as much as possible (e.g. you could use the *getARandomIntBetweenTheTwoIndexes* to implement *getARandomIntBetween0And* eliminating code duplication). Furthermore this class should contain **NO more public methods or variables**.

Player

You will create a class *Player* inside package *cs302.montyhall.concepts.concreteconcepts*

Player will extend the class *AudienceMemberPlayer*, therefore you should not have duplicate instance variables or methods. A player is an audience member that was actually chosen to play the game, therefore not only has a name and a strategy, but now it also has keeps track of:

- a selection of doors that the player is asked to choose from
- a choice of door that the player has made at the moment
- an amount of money that the player will have won at a given point in time
- some randomness that characterizes the particular individual. E.g some people might chose door 2 as their first door, while some others might pick door 3, etc.

You need to create the appropriate instance variables to keep all this information together avoiding duplicate variables that are already defined in *AudienceMemberPlayer*, i.e. you need the following instance variables:

- A *MontyDoor*[] array that will track the door choices available to the player at the moment.
- A *MontyDoor* that will track the players choice of door at the moment.
- An *int* that will track the players winnings at the moment.
- A *MontyRandomness* that will keep track of the player's random behavior.

The class *Player* will have the following public methods:

- A constructor that takes the following arguments (in order) and initializes the appropriate instance variables:
  - A *String* for the players name
  - A *Strategy* for the players strategy
  - A *MontyRandomness* for the players random behavior.

  In addition the constructor will initialize the players door pick and the players door choices both to *null*, and the player's winnings to 0.

- A *getPlayersPick* method that takes no arguments and returns a *MontyDoor* object with the appropriate instance variable value.

- A *getPlayersCurrentChoice* method that takes no arguments and returns a *MontyDoor*[] with the appropriate instance variable value.

- A *getPlayersRandomBehavior* method that takes no arguments and returns a *MontyRandomness* object with the appropriate instance variable value.

- A *getWinnings* method that takes no arguments and returns an *int* with the appropriate instance variable value.

- A *setPlayersCurrentChoices* method with a single *MontyDoor*[] argument that sets the appropriate instance variable value to be the value of the passed argument.

- A setWinnings method with a single int argument that sets the appropriate instance variable value to be the value of the passed argument.

- A pick method that takes a single argument of type *int* and return a *boolean*. Here is what it does:
  It will then look at the index defined by the argument in the instance variable that holds the players choices (i.e. the instance variable of type *MontyDoor*[]) and will set the value of the instance variable that holds the players choice (i.e the instance variable that is of type *MontyDoor*) to be the one of the element at that index.
  In other words, the argument passed in this method will point to a *MontyDoor* element inside the *MontyDoor*[] array, and will set the players pick to be that *MontyDoor* instance.
  If the *int* is outside the available range (i.e. if the *int* given as an argument is not within the range of the array) the method should return *false*.
  Otherwise the method should return *true*.

- A *pick* method that takes NO arguments and returns nothing. It will set the instance variable that holds the players pick (which is of type *MontyDoor*) to a random *MontyDoor* located within the instance variable of type *MontyDoor*[] that holds the players choices.

- A *toString* method that takes no arguments and returns a String that is formatted as follows:
  <whatever an AudienceMemberPlayer toString returns here>. <the players name here> has won <the players winnings here>$ so far by using it.
  e.g.
  Suppose that *Player* George has a *Strategy* of switching 40% of the time that he named "40tude" and he has so far won 100$. Then the *toString* should return:
  George's strategy is "40tude": a strategy in which we switch 40.0% of the time. George has won 100$ so far by using it.

Class *Player* will have NO other public methods or variables. Furthermore you should NOT duplicate variables or methods that already exists in *AudienceMemberPlayer* (or in other classes like *MontyRandomness*). Doing so will result in **0 points for this part of the implementation**.

MontyGameSetup

You will create a class *MontyGameSetup* inside package *cs302.montyhall.concepts.abstractconcepts*

This class will simulate the setup of a Monty game configuration. A Monty game configuration consists of:

- A number of *MontyDoor* object from which the players get to choose.
- A number of *Player* objects that represent the participant players.
- An indicator of which of the available to the players door contains a prize.
- The amount of dollars a player that the prize is worth.

The instance variables you need to create to hold this info are of type:

- *MontyDoor*[]
- *Player*[]
- *int*
- *int*

respectively.

The *MontyGameSetup* Class will contain the following methods:

- A constructor that takes the following arguments:
  - An *int* that represents the number of doors we would like the players to choose from.
  - An array of type *Player* that contains as it's elements all the players that will be playing the Monty game.
  - An *int* that represents the prize amount the players who chose a prize containing door get to win.
  - A *MontyRandomness* object that will be used to create a random winning door assignment.

  The constructor does the following:
  - It creates a new array of type *MontyDoor* and of appropriate size (according to the first argument described above) and populates it with *MontyDoor* objects, only one of which will contain a prize [which one will be determined randomly].
  - It keeps track of the position in the *MontyDoor*[] array that the door with the prize is, and initializes the appropriate instance variable.
  - It sets the freshly created *MontyDoor*[] array as the current choice parameter for each player.
  - It finally initializes the remaining instance variables according to the arguments passed.

- A *getPrizePosition* method that takes no arguments and returns the appropriate instance variable.

- A *getPrizeAmount* method that takes no arguments and returns the appropriate instance variable.

- A *getDoors* method that takes no arguments and returns the appropriate instance variable (of type *MontyDoor*[])

- A *getPlayers* method that takes no arguments and returns the appropriate instance variable (of type *Player*[])

Monty

You will create a class *Monty* inside package *cs302.montyhall.concepts.concreteconcepts*
This class will simulate the Monty person himself. Monty should know everything about the game configuration, i.e. how many doors are there, where the prize is, what is the prize's value, how many players will play and so forth.
Therefore what Monty needs to orchestrate the game is a single *MontyGameSetup* instance variable since a *MontyGameSetup* has all this information.

The *Monty* class will contain the following public methods:

- A constructor that takes a single argument of type *MontyGameSetup* and initializes the instance variable.

- A *setUpANewGame* method that takes as a parameter a single argument of type *MontyGameSetup* and sets the appropriate instance variable to the value of the argument. It returns nothing.

- A *askPlayersToPickADoor* that takes NO parameters and makes all players pick a door (make a choice among their available doors) using their internal random behavior. It returns nothing.

- An *openANonPrizeDoorAmongTheRemainingDoorsForEachPlayer* method that takes a single argument of type *MontyRandomness* and returns nothing. The method should do the following: It should simulate Monty's opening of a door, by removing the door from the players available choices. Thus it should remove a door OTHER than the one that the player has already picked, and furthermore he should never remove the one that has the prize (Monty would never open the door that has the prize behind it). In other words, Monty should remove from each players available *MontyDoor* object a door that does not have the prize behind it and it was not chosen by the player.

- An *askThePlayersIfTheyWantToSwitch* methods that takes a single argument of type *MontyRandomness* and returns nothing. Then it should do the following:
  It should simulate the Monty's questions and the player's response regarding switching.
  The method should create a random threshold of type double between 0 and 100 inclusive.
  Then for each player should either pick a different door among the player's current options if the player's strategy is greater than or equal to the generated threshold, or stay the same if the players strategy is less than the threshold.
  e.g. Suppose we have a player P that has a *Strategy* of switching 100% of the time, and suppose that the threshold generated inside the *askThePlayersIfTheyWantToSwitch* is 67%. Then that player will have to be picking another door.
  Another player P2 that has a *Strategy* of switching 50% of the time will not switch her pick as 50 is less than 67.

- A *calculatePlayersWinnings* method that takes no arguments and updates the winnings for each player. i.e. it adds the prize value to the player's winnings if they had chosen a winning door or 0 otherwise. It returns nothing.

The class *Monty* should contain NO more public methods or variables.

Container Classes:

MontyDoors

You will create a class named *MontyDoors* which will create methods to handle *MonthDoor* arrays. The class should be inside the package: *cs302.montyhall.containers*

The class will contain the following public methods:

- A *removeADoor* methods that takes as arguments a *MontyDoor*[] and an *int* and returns an array that has all the elements of the array in the argument except the element located at the index specified by the second argument (i.e. the array returned will have a smaller by one size.)

- A *getLocationOfPrizeWinningDoor* `that takes as argument a single` *MontyDoor*[] object and return an *int* that denotes the index of the door in the passed argument that has a winning door. If the argument contains MORE than a single prize door, then the method should return -1. Similarly if it contains no prize doors it should return -1.

- A *getPrizeWinningDoor* `that takes as argument a single` *MontyDoor*[] object and returns a *MontyDoor* object that contains the prize. If there are more than one doors or no doors containing the prize the method should return null.

- A *getDoorsThatDoNOTContainThePrize* method that takes `as argument a single` *MontyDoor*[] object and returns a *MontyDoor*[] array that contains all the doors the argument contained except the door that had the prize. If there was no prize door in the argument then it will return the argument unaltered.

- A *getLocationOfDoorWithID* methods that takes as arguments a *MontyDoor*[] and an *int* and returns an returns the index of the element in the passed array that contains a door with the given id. If more than one or no doors in the array has that id, it should return -1.

- A *removeARandomNonPrizeWinningDoor* that takes as arguments a *MontyDoor*[] and a *MontyRandomness* object and returns another *MontyDoor*[] object that contains the same elements with the original argument minus one random element from the argument array that does NOT have a prize. This method should never remove a prize door from the array.

Executable Classes:

PlayMonty

You very last class will be the one that will glue everything else together. This class will contain a single main method.

You will create the *PlayMonty* clas inside package *cs302.montyhall.executable*

The *PlayMonty* will prompt the user for inputting the following:

```
Please enter the number of doors:

Please enter the number of players:

Please enter the prize amount:

Please enter the number of repetitions for the experiment:

Please enter the random seed:

Please enter the random seed change for each iteration:
```

Then for each one of the players it should prompt:

```
Give a name for player <i>:

Give a strategy name for <player name>'s strategy:

Give a strategy percentage for <player name>'s strategy:
```

If the user enters nothing for player then the players name should become Player<i>

If the user enter's nothing for strategy name then the strategy name should become <whatever the player name>'s followed by the strategy percentage:

e.g.:

```
Give a name for player 1:

<enter>

Give a strategy name for Player1's strategy:

<enter>

Give a strategy percentage for Player1's strategy:

46 <enter>
```

The above sequence would create a new player named "Player1" that has a strategy named "Player1's 46.0%"

EDIT:

If the user enters "auto" (without the quotes) for the first player's name, then the program should go ahead and create the appropriate number of players, with names Player1, Player2, …, PlayerN with a random strategy assigned to each one of them, WITHOUT asking the user for player or strategy names anymore.

If the user enters "auto" (without the quotes for the very first prompt) then the program should not ask the user again, and treat all the rest of the questions as if they were answered with a simple enter (as described below)

If the user enters nothing (just hits enter) for the number of doors input prompt then the number of doors should default to 3 (notice that this is the same expected behavior as if the user were to enter "auto". However with "auto" the program will treat ALL subsequent prompts as already answered with entered. With a simple enter it will only consider this question answered with enter).

If the user enters nothing (just hits enter) for the number of players input prompt then the number of players should default to 2.

If the user enters nothing (just hits enter) for the prize amount input prompt then the prize should default to 1.

If the user enters nothing (just hits enter) for the number of repetitions input prompt then the number of repetitions should default to 1000.

If the user enters nothing (just hits enter) for the seed input prompt then the seed should default to 1234567.

If the user enters nothing (just hits enter) for the seed increase input prompt then the seed increase should default to 2.

It will then simulate the experiment for as many times as the user entered, for as many doors as the user entered, and having as many players as the user entered.

Each iteration of the experiment should be creating a new *MontyGameSetup* that has all parameters the same, except the MontyRandomness parameter that should be increasing the seed by the amount of whatever the user entered * iteration number in every iteration.

After the experiment has run for as many times as the user requested, you should print the players using the *toString* method of the *Player* class.

Here is a main that could test your code:

```
public static void main(String[] args)
{
	long seed = 11;
	Percentage percentage1 = new Percentage((float)100);
	Percentage percentage2 = new Percentage((float)50);
	Percentage percentage3 = new Percentage((float)0);
	Strategy strategy1 = new Strategy(percentage1, "Nick is always switching");
	Strategy strategy2 = new Strategy(percentage2, "Carl is switching about half
the time");
	Strategy strategy3 = new Strategy(percentage3, "George is never switching");
	MontyRandomness randomness = new MontyRandomness(seed);
	Player p1 = new Player("Nick", strategy1, randomness);
	Player p2 = new Player("Carl", strategy2, randomness);
	Player p3 = new Player("George", strategy3, randomness);
	Player[] players = {p1, p2, p3};
	MontyGameSetup mgSetup = new MontyGameSetup(3, players, 1, randomness);
	for (int i=0; i<100; i++)
	{
		Monty m = new Monty(mgSetup);
		MontyDoor[] currentChoicesForAllPlayers = mgSetup.getDoors();
		for (Player p: players)
		{
			p.setPlayersCurrentChoices(currentChoicesForAllPlayers);
		}
		m.askPlayersToPickADoor();
		m.openANonPrizeDoorAmongTheRemainingDoorsForEachPlayer(randomness);
		m.askThePlayersIfTheyWantToSwitch(randomness);
		m.calculatePlayersWinnings();
		randomness.reset(seed+i*10);
		mgSetup = new MontyGameSetup(3, players, 1, randomness);
	}
	System.out.println(p1.toString());
	System.out.println(p2.toString());
	System.out.println(p3.toString());
}
```

```
The above will print:

Nick's strategy is "Nick is always switching": a strategy in which we switch 100.0% of the
time. Nick has won 63$ so far by using it.

Carl's strategy is "Carl is switching about half the time": a strategy in which we switch
50.0% of the time. Carl has won 49$ so far by using it.

George's strategy is "George is never switching": a strategy in which we switch 0.0% of the
time. George has won 34$ so far by using it.
```

Tips

1) This is your first serious project in this class. Treat it as serious business. Ask questions online, go to office hours, discuss it with your peers, get help. Do whatever you need to do to complete this assignment as doing this assignment will really teach you and give you practice for the majority of the material we covered so far [which is the material you will see in the midterm exam].
2) The exam is on the Monday after the deadline of the project. Having completed the project will have given you an excellent practice right before the exam.
3) The project looks way scarier than what it is. In fact a big chunk of it you can hopefully already do in just minutes. Namely you should be able to very quickly and easily complete the classes:
   Percentage
   Strategy
   Door
   MontyDoor
   AudienceMemberPlayer
4) Your first "roadblock" will be the *MontyRandomness* class because it will probably sound exotic. However it is not. Take care of the above classes as soon as possible and concentrate on *MontyRandomness* as soon as possible, as completing it really unlocks the way for the rest of the classes.
5) The classes *Player*, *MontyGameSetup* and *Monty* will probably need the majority of your effort to get right and debug. Again, take one at a time, in the order listed and calmly try to implement it. Ask questions and get help as quickly as possible!
6) The *MontyDoors* is a class that does for *MontyDoor* object pretty much what *Arrays* does for *Array* objects. i.e. it provides a bunch of methods to manipulate *MontyDoor* arrays. You should be able to complete it as soon as you have randomness implemented.
7) The very last class, i.e. your executable should be no trouble at all. You already have enough experience with getting user's input and doing things based on it.
8) I will be holding extra office hours next week for help with this project. Take advantage of it!
9) Do NOT panic. You will not be alone in doing this.
10) In order to be able to complete this project you will also have to use the debugging capabilities of eclipse much..much…MUCH more extensively than before.

11) You will simply absolutely fail to complete this project without commenting your code extensively for your own benefit. Giving good variable names and having clean code will be imperative.

Extra Credit

The extra credit this time will reward people that are not procrastinating. Thus submitting the code [i.e. the version of your code that that will be graded] before the deadline will earn you points as follows: Submitting by Sunday midnight will give you full extra credit points.

Commenting Instructions/Guide:

File Header

Requirement: The source file for your CLASS must have a main file header comment located at the beginning of the file containing the following:

```
////////////////////////////////////////////////////////////////////////////////
// Semester:        CS302 Summer 2013
//
// Author:          (your name)
// Email:           (your email address)
// Lecturer's Name:  (name of your lecturer)
//
//
// ANY SOURCE THAT YOU USED BESIDES THE BOOK, TA OR INSTRUCTOR (e.g websites)
//
// Credits:         (list anyone/anything who helped you write your program)
//////////////////////////// 80 columns wide ///////////////////////////////////
```

Class Header

Requirement: Your class must have a header comment located immediately before the class declaration containing the following :

```
/**
* (Write a succinct description of this class here. You should avoid
*  wordiness and redundancy. If necessary, additional paragraphs should
*  be preceded by <p>, the html tag for a new paragraph.)
*
*  <p>Bugs: (a list of bugs and other problems)
*
* @author (your name)
*/
```

Variable Declarations

Requirement: Variable declarations must be commented briefly describing their use. Temporary variables and loop counters do not need to be commented.

Other Comments

Requirement: Use comments within the body of methods to:

- highlight the major steps of your algorithm
- explain long calculations or conditions

- clarify convoluted or unusual code
- mark locations where you suspect a bug may exist
- mark locations where improvements or enhancements are planned

Do not use comments in methods to explain things that are obvious to programmers. For example, it is not useful to provide comments such as "this assignment statement assigns 22 to the variable x". Instead use comments to point out things that are not immediately evident or to highlight sections of code.

Grading

Correct implementation of the following classes will give you 5 points for each:

- Percentage
- Strategy
- Door
- MontyDoor
- AudienceMemberPlayer
- PlayMonty

for a total of 30 points.

Correct implementation of the following classes will give you:

- MontyRandomness (10)
- Player (15)
- MontyGameSetup (10)
- Monty (15)
- MontyDoors (10)

for a total of 60 points.

Commenting/Code style 10 points.

How to submit

You will submit your project as a zipped file.
Follow the instructions online as to how to export your project in a zip file.
Submit your exported project.
Make sure to name your exported project file as :
lastname_studentID_cs302_Project3.zip

For example:

Pappas_123456_cs302_Project3.zip

I cannot stress enough how important it is to start this program **immediately**.

Good luck!